PROJECT-BASED LEARNING

PHASE-II EVALUATION

Secure CBT: Hierarchical Key-Chained Question Paper Delivery & Leak-Traceability System

Team ID: DSCPP-III-2026-T145

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID:	DSCPP-III-2026-T145
Team Name:	Secure CBT
Semester:	3rd
Section:	BTECH
Domain:	Cybersecurity & DSA

Team Members

1. Naman – 2510013438
2. Aayush Gupta – 2510370185
3. Kushagra Bhushan – 2510012263

Mentor

Mr. Kartikey Arora

Interactions completed: 4

Phase-I Recap & Feedback Incorporated

Phase-I Status

Proposed the 3-phase Secure CBT architecture and implemented initial classes (`Question`, `QuestionBank`, `TopicIndex`, `PaperGenerator`) with Huffman coding and basic XOR key derivation.

Feedback Received

- Upgrade basic XOR keys to standard AES-256 encryption
- Add a CSV validator and teacher upload screen
- Implement Phase 2 LCRS tree, AVL registry, and SHA-256 log

Changes Incorporated

- Integrated OpenSSL AES-256-GCM and HKDF-SHA256
- Built `csvchecker.h` and the live web upload portal
- Completed `AVLRegistry.h`, `LCRSTree.h`, and `AuditLog.h`

Problem Statement & Finalized Objectives

Problem Statement

Standard CBT systems send an identical encrypted paper to every exam centre, making it hard to trace a leaked file without external logs. **Secure CBT** embeds traceability directly into the encryption chain so each node receives a uniquely keyed copy.

Finalized Objectives

1. **Phase 1 – Paper Setting:** Validate uploaded CSV question banks, sample balanced papers via Fisher–Yates shuffle, compress with Huffman coding, and encrypt with AES-256.
2. **Phase 2 – Hierarchy & Registry:** Represent the N-ary distribution network using an LCRS tree and verify nodes in $O(\log n)$ via an AVL tree.
3. **Phase 2 – Layered Delivery & Audit:** Broadcast via BFS with per-hop key chaining and record every transfer in a SHA-256 hash chain.
4. **Phase 3 – Exam Centre:** Enforce time-locked decryption and trace any leaked ciphertext via the AVL registry.

Updated System Architecture / Workflow



Major Components

- **Phase 1:** CSV checker, QuestionBank, Huffman, PaperStorage
- **Phase 2:** AVLRegistry, LCRSTree, KeyManager, AuditLog
- **Web Portal:** Teacher upload and distribution dashboard

Changes Since Phase-I

- Upgraded from XOR to OpenSSL AES-256-GCM
- Added 5-level layered encryption across hierarchy hops
- Added binary `.scbt` package format and live web portal

Module Design: Phase 1 – Paper Setting

Phase 1 Modules

1. `csvchecker.h`: Validates CSV headers and rows
2. `QuestionBank.h` & `TopicIndex.h`: Hash-map storage
3. `PaperGenerator.h`: Fisher–Yates sampling without repeats
4. `HuffmanCompressor.h`: Min-heap Huffman compression
5. `PaperStorage.h`: Saves encrypted `paper.scbt` package

Functional Requirements

- Reject malformed CSVs with exact row and field errors
- Pick balanced questions per topic with zero duplicates
- Compress paper before encryption (36.8% size reduction)
- Store the root key (`paper.key`) separate from `paper.scbt`

Module Design: Phase 2 – Secure Distribution

Phase 2 Modules

1. `AVLRegistry.h`: Self-balancing AVL tree for node lookup
2. `LCRSTree.h`: Left-Child Right-Sibling tree + BFS broadcast
3. `KeyManager.h`: Child key derivation & AES-256 encryption
4. `AuditLog.h`: SHA-256 hash-chained transfer log

Functional Requirements

- Verify node status in $O(\log n)$ and block unauthorized nodes
- Broadcast strictly parent $\rightarrow$ child level-by-level via BFS queue
- Re-encrypt payload at each hop with derived child key
- Detect any modified transfer record via SHA-256 chain check

Code Pipeline – Phase 1 & Phase 2

Phase 1 – Paper Setting Pipeline

`sample-questions.csv` ⟶ `checkCsv()` ⟶ `QuestionBank` & `TopicIndex` ⟶ `PaperGenerator` ⟶ `HuffmanCompressor` ⟶ `KeyManager::encrypt()` ⟶ `paper.scbt` (3,416 bytes)

Phase 2 – Secure Distribution Pipeline

`paper.scbt` ⟶ `AVLRegistry::isRegistered()` ⟶ `LCRSTree::bfsBroadcast()` (Board → State → District → City → Centre) ⟶ `AuditLog::append()` ⟶ `unwrapAtNode()`

Each hop derives a child key from its parent key and wraps the payload in a new AES-256-GCM layer (+28 bytes per hop: 12B IV + 16B auth tag).

Technology Stack & Development Methodology

Technology Stack

- **Programming:** C++17
- **Cryptography:** OpenSSL (AES-256-GCM, SHA-256)
- **Data Structures:** AVL tree, LCRS tree, Min-Heap, Hash Maps
- **Web Portal:** Web Upload Portal
- **Version Control:** Git / GitHub

Development Methodology

1. Requirement Analysis
2. Data-Structure & Class Design
3. Modular Header Implementation
4. Phase 1 $\rightarrow$ Phase 2 Pipeline Integration
5. Tamper & Encryption Verification

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	AVL Tree, LCRS Tree, BFS Queue, Min-Heap, Hash Tables, Hash Chain	`AVLRegistry.h`, `LCRSTree.h`, `HuffmanCompressor.h`, `AuditLog.h`
OOPs with C++	Classes, Encapsulation, Constructors/Destructors, STL, File Streams	`Question.h`, `QuestionBank.h`, `PaperGenerator.h`, `KeyManager.h`
Algorithms	Huffman Coding, Fisher–Yates Shuffle, Level-Order BFS Traversal	`HuffmanCompressor.h`, `PaperGenerator.h`, `LCRSTree.h`
Cryptography	AES-256-GCM Encryption, Key Chaining, SHA-256 Hashing	`KeyManager.h`, `PaperStorage.h`, `AuditLog.h`

Development & Implementation Progress

Module	Status	Work Completed
Phase 1 – CSV & Paper Generator	**Completed**	`csvchecker.h`, `QuestionBank.h`, `TopicIndex.h`, `PaperGenerator.h`
Phase 1 – Huffman & Encryption	**Completed**	`HuffmanCompressor.h`, `KeyManager.h`, `PaperStorage.h` (`.scbt`)
Phase 2 – AVL & LCRS Broadcast	**Completed**	`AVLRegistry.h` ($O(\log n)$ lookup), `LCRSTree.h` (BFS broadcast)
Phase 2 – SHA-256 Audit Log	**Completed**	`AuditLog.h` with hash-chain tamper detection
Phase 3 – Centre Time-Lock & Trace	**Pending**	Time-locked decryption and leaked ciphertext matching

Working Prototype: Phase 1 – Paper Setting

Implemented Features

- **Live Web Portal:** `secure-cbt.onrender.com`
- **CSV Validation:** 78 valid questions loaded; 39 selected (221 marks)
- **Compression:** 42,872 bits → 27,097 bits (3,388 bytes)
- **Encrypted Output:** Saves `paper.scbt` (3,416 bytes)

Terminal Output (`./secure_cbt data/sample-questions.csv`)

```
===== PHASE 1: PAPER SETTING =====
Checked file: sample-questions.csv
Valid questions in file: 78
Paper will have 39 questions, 221 marks.
----- Assembled paper (5359 characters) -----
HuffmanCompressor created
Original size : 42872 bits
Compressed size: 27097 bits (3388 bytes)
Root key generated (fingerprint: c3faaa41)
Data encrypted with AES-256-GCM
Encrypted paper size: 3416 bytes
Saved encrypted paper to paper.scbt (verified by reading it back)
===== PHASE 1 COMPLETE =====
```

Working Prototype: Phase 2 – Secure Distribution

Implemented Features

- **AVL Registry:** 15 nodes registered (height 5, balanced)
- **BFS Broadcast:** All States first (Hops 01–03), then Districts, Cities, and Centres
- **Centre Unwrap:** Peels all 5 layers at `CTR-HR-01`
- **Audit Log:** Catches tampering at Entry 3

Terminal Output (`./secure_cbt_phase2`)

```
Loaded Phase 1 package: paper.scbt (3416 encrypted bytes)
Total nodes registered in AVL tree : 15 (height: 5, balanced: YES)
  Hop 01 | BOARD -> ST-01 | Delivered (3444 bytes)
  Hop 02 | BOARD -> ST-02 | Delivered (3444 bytes)
  Hop 03 | BOARD -> ST-08 | Delivered (3444 bytes)
  Hop 04 | ST-01 -> ST-01-D01 | Delivered (3472 bytes)
  Hop 07 | ST-01-D01 -> ST-01-D01-CT | Delivered (3500 bytes)
  Hop 10 | ST-01-D01-CT -> CTR-AP-01 | Delivered (3528 bytes)
  Hop 11 | ST-01-D01-CT -> CTR-AP-02 | Blocked (Not Registered)
  Hop 14 | ST-08-D01-CT -> CTR-HR-99 | Blocked (Not Registered)
Peel all 5 layers at CTR-HR-01: VERIFIED
Audit chain verification : CHAIN INTACT
Audit chain after tamper : CHAIN BROKEN AT ENTRY 3
```

Testing, Results & Validation

Module	Expected Result	Observed Result	Status
`csvchecker.h`	Accept valid CSV; flag bad rows	78 rows loaded; row errors caught	**Pass**
`HuffmanCompressor.h`	Lossless compression	42,872 → 27,097 bits (36.8% saved)	**Pass**
`KeyManager.h`	Reject wrong key or modified byte	Tampered ciphertext rejected	**Pass**
`AVLRegistry.h`	Balance sorted nodes; block Pending	Height 5; unauthorized nodes blocked	**Pass**
`LCRSTree.h` & `AuditLog.h`	5-layer unwrap & log tamper check	Unwrapped at centre; Entry 3 tamper caught	**Pass**

GitHub Repository & Development Activity

Repository Information

- **GitHub:** `Kushagratriest/Secure-CBT`
- **Live Demo:** `secure-cbt.onrender.com`
- **README.md:** Updated with architecture & workflow
- **Contributors:** Naman, Kushagra Bhushan, Aayush Gupta
- **Branch:** `main`

Development Activity

- Organized into `common/`, `phase1/`, and `phase2/`
- Regular commits across Phase 1 and Phase 2 modules
- Meaningful commit messages per feature
- Individual module contributions tracked on GitHub
- Project documentation and reports updated

Individual Contribution

Member (GitHub)	Role	Technical Contribution (GitHub Verified)
Naman – 2510013438 (namangoyal1908)	Team Lead	Created `Question.h`, `QuestionBank.h`, `TopicIndex.h`, `csvchecker.h`, initial `PaperGenerator.h` & `KeyManager.h`, and `LCRSTree.h`
Aayush Gupta – 2510370185 (aayush-pixel77)	Developer	Created `AuditLog.h` (SHA-256 chain) & Phase 1 `main.cpp`; upgraded `KeyManager.h` (AES-256-GCM) & `PaperGenerator.h` (Fisher–Yates)
Kushagra Bhushan – 2510012263 (Kushagratriest)	Developer	Created `HuffmanCompressor.h`, `PaperStorage.h` (`.scbt`), `AVLRegistry.h`, `server.cpp`, `test_audit_log.cpp`, and Web Portal

Challenges, Phase-III Roadmap & References

Challenges & Solutions

- **CSV Commas:** Fixed with quoted-field parser in `csvchecker.h`
- **Huffman Padding:** Fixed by storing exact `bitCount` in `paper.scbt`

Phase-III Roadmap

1. Time-locked decryption at exam centre (`TimeLock.h`)
2. Automated leak-source tracing via `AVLRegistry`
3. Dijkstra & MST network routing simulation

References

1. Newslaundry (2024), Indian paper-leak report
2. Cormen et al., *Introduction to Algorithms*
3. NIST FIPS 197 (AES) & FIPS 180-4 (SHA-256)
4. OpenSSL 3.0 Official Documentation

Thank You
Questions & Suggestions